AI Resume Analyzer & Mock Interview Platform

Theme: Career & Job Tech

AI Resume Analyzer & Mock Interview Platform

"Because every candidate deserves a fair shot — and honest feedback before the real interview."

Real-World Context

Every year, millions of students and job-seekers apply for jobs, but most of them never get real feedback on why they were rejected. A resume might get rejected by an automated screening system before a human even reads it. Many candidates don't know how to structure their resume, what keywords matter for a specific job role, or how they actually perform in an interview until it's too late — the interview is already over.

Career coaching, resume reviews, and mock interview practice with real feedback are expensive and not accessible to everyone. Most people are left guessing:

- "Is my resume good enough for this job?"
- "Why do I keep getting rejected at the resume screening stage?"
- "How do I actually sound and perform in an interview?"

There is a clear need for an accessible, intelligent tool that gives people honest, actionable feedback — for free, anytime, without needing to book a coach.

The Problem Statement

Build an **AI Resume Analyzer & Mock Interview Platform** — a web-based application where a user can upload their resume and a target job description, get a detailed analysis of how well their resume matches the role, and then practice for the interview through an AI-driven mock interview experience that gives real feedback on their answers.

This should feel like having a personal career coach available 24/7, not just a static checklist.

Objective

By the end of the hackathon, your team must deliver a **working, full-stack application** that can:

1. Let a user create an account and securely log in.
2. Allow a user to upload their resume (PDF/DOCX) and paste or upload a target job description.
3. Analyze the resume against the job description and generate a clear match score with specific, actionable feedback.
4. Run an AI-powered mock interview based on the resume and job role, and give the user feedback on their answers.
5. Track the user's progress and history over multiple attempts.

Key Requirements (Must-Haves)

Your solution must include **all** of the following — plan your team's time across all days carefully, this is meant to be genuinely challenging:

1. User Accounts & Authentication

- Secure sign-up/login system.
- Each user's resumes, analysis history, and interview sessions must be private to their account.

2. Resume Upload & Parsing

- Accept resume uploads in PDF and/or DOCX format.
- Extract and structure the resume content (name, skills, experience, education, projects, etc.) from the uploaded file — this parsing step is a core technical challenge, not a trivial text extraction.

3. Job Description Matching Engine

- Allow the user to paste or upload a job description.
- Compare the resume content against the job description and generate:
  - A **match score** (e.g., percentage or rating).
  - A list of **missing or weak keywords/skills**.
  - Specific, actionable suggestions on how to improve the resume for that particular role (not generic advice).
- You are expected to **research and integrate a suitable third-party AI/NLP API** to power this analysis (for example, a language model API, resume-parsing API, or NLP/text-similarity API). Evaluating available options and picking the one that fits your solution is part of the challenge. Clearly document in your README **which API/service you used, why you chose it, and how it's integrated.**

4. AI-Powered Mock Interview Module

- Generate relevant interview questions based on the user's resume and the target job role (a mix of technical and behavioral questions).
- Allow the user to answer questions via text input (voice input can be a bonus feature).
- Use an AI/NLP service to evaluate the user's answers and give constructive feedback — covering things like clarity, relevance to the question, and completeness of the answer.
- Give an overall performance summary at the end of the mock interview session.

5. Dashboard & History

- Show the user their past resume analyses and mock interview sessions.
- Display progress over time (e.g., improving match scores or interview performance across attempts).

6. Database Integration

- Persistently store user accounts, uploaded resumes/parsed data, job descriptions, analysis results, and interview session history.

7. Responsive, Clean UI

- Should work smoothly on both desktop and mobile.
- Design should feel encouraging and professional — this is a tool meant to build a user's confidence, not add to their stress.

8. Error Handling

- Handle cases like unsupported file formats, corrupted uploads, API failures, and empty/invalid job descriptions gracefully — never leave the user stuck with no explanation.

Challenges

- **Voice-Based Mock Interview**: Let users answer interview questions by speaking, using speech-to-text, and even analyze tone/confidence from voice.
- **ATS Compatibility Checker**: Simulate how an Applicant Tracking System (ATS) might parse the resume, and flag formatting issues that could cause rejection.
- **Resume Builder/Rewriter**: Suggest or auto-generate improved resume bullet points based on the job description.
- **Industry-Specific Question Banks**: Tailor mock interview questions based on the specific industry or role (e.g., software engineering vs. marketing vs. finance).
- **Video Mock Interview with Body Language Feedback**: Use webcam input to give basic feedback on eye contact, posture, or expressions during practice.
- **Peer/Recruiter Sharing Mode**: Let users generate a shareable feedback report they can send to a mentor or recruiter for additional review.

Deliverables

Each team must submit:

1. **Fully functional deployed application** (deployment strongly recommended) — frontend + backend + database.
2. **GitHub repository** following the naming and structure guidelines from the General Instructions (HackInMotion-TeamCode), including:
    - architecture-diagram.png
    - api-documentation.md
    - presentation.pptx
    - Complete README.md (as per Section 8 of General Instructions — including which third-party API/service was used and how)
3. **Live demo** showing a real resume being analyzed against a job description, and a live/recorded mock interview session with feedback.
4. **Product Pitch** (for finalists) covering the problem, solution, tech stack, real-world impact, and future scope.

Suggested Tech Stack (Flexible — choose what your team is confident with)

- **Frontend**: React.js / Next.js
- **Backend**: Node.js (Express) / Python (Django/Flask/FastAPI) / Java (Spring Boot) / .NET (ASP.NET Core)
- **Database**: MongoDB / PostgreSQL / MySQL / Firebase
- **Third-Party AI/NLP Service**: Research and select a suitable **AI/language model or NLP API** for resume analysis and interview answer evaluation — evaluating multiple options and picking one that fits your solution is part of the challenge.
- **File Parsing**: Look into libraries/services for extracting text from PDF/DOCX files.
- **Authentication**: JWT / Firebase Auth / OAuth
- **Deployment**: Vercel / Netlify / Render / Railway / AWS / Azure